

ESTRUCTURAS

En algunas aplicaciones los datos están relacionados entre sí, pero son de diferente tipo, y por lo tanto no es posible utilizar un vector para relacionarlos y se deberían usar varias variables con diferente nombre. Si quisiéramos guardar los datos de una agenda telefónica veríamos que todos los datos son de diferente tipo, y es por eso que no sería posible utilizar un vector para agruparlas.

Esto se soluciona mediante el uso de un tipo de variable llamado estructura. En ésta los datos no tienen que ser necesariamente del mismo tipo y todos ellos se agrupan utilizando el mismo nombre.

A cada dato particular en la estructura se lo denomina campo y cada campo recibe un nombre para que sea posible referenciarlo.

Es decir, una estructura está compuesta por campos que son en sí distintas variables agrupadas por un mismo nombre.

Cabe destacar que una estructura es un tipo de variable definido por el usuario. Por lo tanto para poder utilizarla primero hay que declararla como tal.

Como es una variable se lo puede utilizar para realizar la operación de asignación. Es decir, se puede asignar una estructura a otra, siempre que sean del mismo tipo.

Los campos de las estructuras pueden ser de cualquier tipo: int, char, float, long, etc., o también pueden ser vectores u otras estructuras.

La estructura debe ser definida. Puede ser definida en forma global, antes del comienzo de todas las funciones, y en este caso la verían todas las funciones, o en forma local, dentro de la definición de una función, en este último caso solo sería vista por la función en la cual ha sido definida, al igual que las reglas correspondientes a la definición de cualquier tipo de variable permitida en "C".

Estas permiten una gran flexibilidad en las posibilidades de utilización pero hay que tener cuidado en su manipulación, porque como son variables complejas puede suceder que se cometan errores cuando se las utiliza.

DECLARACIÓN

Para poder utilizar una estructura se la debe declarar, indicando el tipo y nombre de cada campo que la va a constituir.

Hay tres posibles formas, que son las siguientes:

```
struct etiqueta {   tipo campo_1;  
                  tipo campo_2;\  
                  tipo campo_3;  
                  ...  
                  ...  
                  tipo campo_n;} lista_de_variables;
```

Esta es la forma más general de declararla, en ella se debe utilizar la palabra reservada 'struct', luego un nombre del tipo llamado etiqueta, a continuación se utilizan llaves de apertura y de cierre ({ }), éstas sirven para encerrar los campos que la integrarán, y por

último se definen las variables que tendrán el tipo de la estructura definido entre las llaves, estas variables se declaran dentro de la lista_de_variables.

En general esta forma se la utiliza para declararlas de forma global o local pero solamente en una única función.

Otra forma de declarar una estructura puede ser omitiendo a la etiqueta, pero en este caso se debe necesariamente incluir la lista_de_variables. De esta forma si se omitiese esta lista no se puede utilizar, porque no se la podrá identificar de ninguna forma.

```
struct { tipo campo_1;  
    tipo campo_2;  
    tipo campo_3;  
    ...  
    ...  
    tipo campo_n;} lista_de_variables;
```

Esta forma se utiliza en muy pocas circunstancias, debido a su poca flexibilidad.

Otra manera de declararla es omitir la lista_de_variables, este formato es el más utilizado debido a que al declarar la etiqueta del tipo, éste se convierte en un nuevo tipo de variable que podrá ser utilizado en el lugar del programa que se desee, es decir, en cualquier función, solo con declarar el tipo cuando se la desee utilizar, y también es mas fácil de interpretar cuando se lee un programa.

```
struct etiqueta { tipo campo_1;  
    tipo campo_2;  
    tipo campo_3;  
    ...  
    ...  
    tipo campo_n;};
```

Solo basta en este caso, declarar a las variables de la siguiente forma:

```
struct    etiqueta    lista_de_variables;
```

Además, es muy útil cuando se utilizan como parámetros de una función.

Utilizando esta forma se puede declarar el tipo en forma global o local.

Ejemplo

```
#include <stdio.h>  
struct dato { int legajo;  
              float nota1,nota2,nota3;} alumno;  
float calcprom(void);  
void main(void)  
{  
    struct dato mejor;  
    -----  
    -----  
    -----  
}
```

```
float calcprom(void);
{
    struct dato nuevo;
    -----
    -----
    -----
}
```

En las líneas de programa anterior se puede observar que se definió una estructura global llamada “dato” y en esa definición se declaró una variable “alumno” que es global al programa. En las funciones “main” y “calcprom” se declaran dos variables locales, la denominada “mejor” en “main” y la otra “nuevo” en “calcprom”. Cada una de ellas es local a su propia función (solo es vista por la función en la que fue definida).

Como todas las variables tienen la misma etiqueta son del mismo tipo, por lo tanto “alumno” y “mejor” o “alumno” y “nuevo” pueden ser asignadas entre sí.

NOTA: LAS LÍNEAS DE GUIONES QUE APARECEN EN LAS FUNCIONES DEBEN SER CONSIDERADAS COMO LAS SENTENCIAS DE UNA FUNCIÓN, NO PERTENECEN AL LENGUAJE “C”, SOLO HAN SIDO UBICADAS PARA RESALTAR A LAS ESTRUCTURAS QUE SIRVEN PARA EJEMPLIFICAR.

```
#include <stdio.h>
struct { int legajo;
        float nota1,nota2,nota3;} alumno;
float calcprom(void);
void main(void)
{
    struct { int legajo;
            float nota1,nota2,nota3;}mejor;
    -----
    -----
    -----
}
float calcprom(void);
{
    struct { int legajo;
            float nota1,nota2,nota3;} nuevo;
    -----
    -----
    -----
}
```

En el anterior se omiten las etiquetas, por lo tanto para poder utilizar estructuras que tienen la misma forma se deberá definir en cada función o en forma global dicho tipo de estructura.

Como estas han sido definidas por separado no son del mismo tipo y por lo tanto todas las variables no se pueden asignar entre sí. Si se quisieran asignar todos los campos de una estructura a otra se deberá realizar individualmente por campo.

Otra desventaja que posee es que no se pueden declarar otras variables por separado en las funciones existentes o tampoco declarar nuevas variables, en una nueva función, con ese formato.

```

#include <stdio.h>
struct dato { int legajo;
              float nota1,nota2,nota3;};
float calcprom(void);
void main(void)
{
    struct dato mejor;
    -----
    -----
    -----
}
float calcprom(void);
{
    struct dato nuevo;
    -----
    -----
    -----
}

```

Se puede observar en el caso anterior la ventaja que posee definir una estructura de forma global, en este caso se omitió la lista de variables en la definición. Si se deseara declarar una variable de ese tipo en forma global, solo bastaría con escribir el tipo de la forma “struct dato” y a continuación los nombres de las variables que le correspondieran.

OPERADOR MIEMBRO Ó PUNTO (.)

Cuando se quiere hacer referencia a un campo de la estructura se debe utilizar el operador punto (.).

Si se quiere utilizar un campo para asignarle un valor o realizar cualquier operación en éste se debe mencionar la variable asociada a la estructura luego el operador punto y a continuación el campo al cual se quiere hacer referencia.

Es decir, cuando quiero utilizar un campo de una estructura debo emplear este operador para poder hacerlo.

Ejemplo

```

/* Ejemplo de la utilización del operador punto */
#include <stdio.h>

/* Definición de la estructura a utilizar */
struct productos { int codigo,cantidad;
                  float precio;};

void main(void)
{
    struct productos lista;
    /* Asignación de valores a los campos de la estructura */
    lista.codigo=104;
    lista.cantidad=5;
    lista.precio=54.39;
}

```

```

/* Impresión de los campos en pantalla */
printf("\n%d\t%d\t%.2f$", lista.codigo, lista.cantidad, \
      lista.precio);
}

```

En el ejemplo indicado se define una estructura llamada “productos” que contiene 3 campos, en la función “main” se declara una variable de este tipo llamada “lista”. Luego se asignan valores a cada uno de los campos de la estructura. Este procedimiento es totalmente válido, debido a que cada uno de los campos es una variable de un tipo lícito en “C”, por lo tanto se puede realizar dicho procedimiento. Dentro de esta función se desea imprimir en pantalla los campos de la estructura utilizando el operador punto para ello, dentro de la función “printf”.

ASIGNACIÓN DE ESTRUCTURAS

Como una estructura es una variable de tipo definido por el usuario, se puede asignar una estructura a otra, siempre y cuando ambas sean del mismo tipo. Esto se realiza indicando el nombre de una estructura seguido por el operador igual (=) y luego el nombre de la estructura que se le asignara a la primera. En síntesis de la misma forma que se signan dos variables de tipo estándar (int, float, etc.). Solo hay que tener cuidado en que todos sus campos sean iguales.

Ejemplo

```

#include <stdio.h>
/* Definición de la estructura */
struct lista { int art,cant;
              float prec;};
void main(void)
{
    /* Declaración de dos variables del tipo estructura lista *
     * valcom significa valor de compra                               *
     * valven significa valor de venta                                */
    struct lista valcom,valven;
    /* Asignación de valores a los campos de la variable valcom */
    valcom.art=56;
    valcom.cant=3;
    valcom.prec=23.59;
    /* Asignación del valor de la variable valcom *
     * a la variable valven                               */
    valven=valcom;
    /* Verificación de los valores de los campos de cada variable */
    printf("\n\n%d\t%d\t%.2f",valcom.art,valcom.cant,valcom.prec);
    printf("\n\n%d\t%d\t%.2f",valven.art,valven.cant,valven.prec);
    /* Adición de un 15% al precio de venta */
    valven.prec*=1.15;
    /* Verificación de los valores de los campos de cada variable */
    printf("\n\n%d\t%d\t%.2f",valcom.art,valcom.cant,valcom.prec);
    printf("\n\n%d\t%d\t%.2f",valven.art,valven.cant,valven.prec);
}

```

Aquí encontramos la utilización de la asignación de una estructura a otra, permitiendo la copia de todos los campos de una a otra en una única sentencia. Si usted compila y ejecuta este programa podrá verificar que ambas variables contienen los mismos valores en todos sus campos. Luego para poder verificar que no se modifican automáticamente se realiza una operación sobre un campo de la variable “valven” y a continuación se imprimen en pantalla ambas variables.

INGRESO DE VALORES EN UNA ESTRUCTURA

Para ingresar un valor en una estructura se utilizan las mismas funciones que para cualquier otro dato en una variable. Es decir, cuando se quiere ingresar un carácter se deben utilizar las funciones “getch”, “getche” o “getchar”. En el caso de querer ingresar un valor numérico tanto sea entero como de punto flotante se utilizara la función “scanf” con el formato de control adecuado en la misma; si se desea ingresar un valor numérico en un campo de tipo “char” se debe utilizar el formato de control “%d” en el caso de que el mismo este en base decimal, para ingresarlo en formato hexadecimal deberá utilizar “%x” o “%X” y en el caso de que esté en base octal se deberá utilizar “%o”.

Ejemplo

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
struct valor { char x;
               int j;
               float l;};
void main(void)
{
    struct valor dato;
    clrscr();
    printf("\n\n\n\t\tIngrese los siguientes datos: ");
    printf("\n\nUn carácter alfabético : ");
    dato.x=getche();
    printf("\n\nUn número entero : ");
    scanf("%d",&dato.j);
    printf("\n\nUn número real : ");
    scanf("%f",&dato.l);
    clrscr();
    printf("Usted ingresó los siguientes valores\n\n");
    printf("Un carácter %c, cuyo valor ASCII es %d,\n",
           dato.x,dato.x);
    printf("en formato hexadecimal es %X\n\n",dato.x);
    printf("Un número entero %d, ",dato.j);
    printf("que en formato hexadecimal es %X\n\n",dato.j);
    printf("El valor real es %f\n\n",dato.l);
}
```

Aquí encontramos las diferentes formas de poder ingresar datos de diferentes tipos dentro de una estructura.

ESTRUCTURAS QUE CONTIENEN VECTORES

El único cuidado que hay que tener es que el subíndice del vector corresponde a un campo y debe incluirse al finalizar el nombre del campo y no antes de éste.

Como dijimos anteriormente una estructura puede contener cualquier tipo de variables en sus campos, no hay ningún tipo de restricción para incluir un vector dentro de una de ellas.

Ejemplo

```
#include<stdio.h>
#include<string.h>
#include<conio.h>
/* Define a la estructura "pers" */
struct pers { char nom[20],apel[20];
              char edad;
              float peso,altura;};
void main(void)
{
    /* Declara una variable del tipo struct pers llamada "habit" */
    struct pers habit;
    /* Declara una variable auxiliar */
    float aux;
    clrscr();
    /* Ingreso de los datos a la estructura */
    printf("\n\t\tINGRESE LOS DATOS DE LA PERSONA\n\n");
    printf("Nombre : ");
    gets(habit.nom);
    printf("\nApellido : ");
    gets(habit.apel);
    printf("\nEdad : ");
    /* Ingresa el valor de la edad como valor numérico utilizando *
    * el formato de control "%d" y no el formato de control "%c" *
    * que se utilizaría para ingresar un carácter desde el *
    * teclado sin importar que la variable a la que se ingrese *
    * dicho valor sea del tipo "int" o del tipo "char" debido a *
    * que ambos tipos son enteros */
    scanf("%d",&habit.edad);
    printf("\nPeso : ");
    scanf("%f",&aux);
    habit.peso=aux;
    printf("\nAltura : ");
    scanf("%f",&aux);
    habit.altura=aux;
    printf("\n\n\t\tPresione una tecla para continuar");
    /* Espera para visualizar los datos en pantalla */
    getch();
    clrscr();
    /* Impresión de los datos en pantalla con un formato distinto */
    printf("\n\n\t\tDatos de %s,%s",habit.apel,habit.nom);
    printf("\n\n\n\t\tEdad : %d\t\t",habit.edad);
    printf("Altura : %1.2f\t\tPeso : %2.2f",habit.altura,habit.peso);
}
```

En la estructura definida se puede observar que existen dos vectores como campos uno llamado “nom” y el otro “apel” estos son de tipo “char” para ser utilizados como strings. Ello no invalida que se puedan utilizar otro tipo de vectores dentro de una estructura. Luego se realiza el ingreso de los datos, aquí podemos observar dos características particulares que debemos analizar. La primera es en el ingreso del dato de la edad, en el se utiliza el formato “%d” en la cadena de control de la función “scanf” ello se realiza para poder ingresar el dato con formato numérico por más que la variable sea de tipo “char”, ésta también es de tipo entero al igual que “int”. El otro caso particular que hay que analizar es el que se utiliza una variable auxiliar para ingresar los valores de peso y altura, esto se debe a que la función “scanf” tiene un “bugs” para esta característica cuando se utiliza una variable de tipo “float” como campo de una estructura.

VECTORES DE ESTRUCTURAS

Se puede declarar a una estructura como un vector. Esto se realiza declarando una variable del tipo de la estructura como se declara a un vector. Hay que tener en cuenta solo que la posición del vector debe incluirse a continuación del nombre de la estructura y antes del operador punto. En este caso se reservara en memoria la cantidad de elementos del vector, pero cada elemento corresponde al tamaño de una estructura. Cabe destacar que cuando se quiere ingresar un campo de tipo float a una variable que es un vector de estructuras, utilizando la función scanf, se debe realizar un procedimiento particular, debido a que esta función tiene un error para este caso especial, se debe realizar el ingreso de una variable auxiliar utilizando scanf y luego asignarle el contenido de esta variable al campo de la estructura. Este inconveniente no ocurre siempre, por lo tanto al no tener la seguridad de un correcto funcionamiento debemos tomar dicha precaución.

Ejemplo

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#define N 100
#define PANT 20
/* Define la estructura a utilizar */
struct stock { int art,cant;
               char desc[30];
               float precio;};

void main(void)
{
    int i,j;
    float faux;
    /* Declara un vector de estructuras y una variable auxiliar */
    struct stock prod[N],aux;
    clrscr();
```



```

/* Lazo de ingreso de datos */
for(i=0;i<N;i++)
{
    printf("artículo[%d]\t",i+1);
    scanf("%d",&prod[i].art);
    printf("cantidad\t");
    scanf("%d",&prod[i].cant);
    /* Limpia el contenido del buffer de teclado */
    fflush(stdin);
    printf("descripción\t");
    gets(prod[i].desc);
    printf("precio\t");
    /* Se ingresa en una variable auxiliar el valor en punto *
     * flotante y luego se le asigna al campo correspondiente *
     * para evitar el bugs que posee "scanf" */
    scanf("%f",&faux);
    prod[i].precio=faux;
    printf("\n");
}
clrscr();
/* rutina de ordenamiento del vector */
for(i=0;i<N-1;i++)
    for(j=i+1;j<N;j++)
        if(prod[j].cant>prod[i].cant)
        {
            /* Intercambio de los valores a ordenar */
            aux=prod[j];
            prod[j]=prod[i];
            prod[i]=aux;
        }
/* Rutina de impresión de los datos en pantalla dividido *
 * por páginas */
for(i=0;i<N;i+=PANT)
{
    printf("Artículo\tCantidad\tDescripción\tPrecio\n");
    for(j=0;(j<PANT)&&(i+j<N);j++)
    {
        printf("%d\t\t%d",prod[i+j].art,prod[i+j].cant);
        printf("\t\t%s\t\t%f.2\n",prod[i+j].desc,\
            prod[i+j].precio);
    }
    printf("\n\tPresione una tecla para continuar");
    getch();
    clrscr();
}
}

```

Se utiliza un vector de estructuras para guardar los datos del stock de una empresa con los siguientes ítems: número de artículo, cantidad de unidades y la descripción del mismo.

Luego se realiza el ordenamiento por cantidad de éstos y por último se los imprime en la pantalla teniendo cuidado de evitar el scrolling de la misma.

Hay que poner de manifiesto que se debe utilizar una variable auxiliar para ingresar el valor “float” debido al error que posee la función “scanf”.

ESTRUCTURAS DENTRO DE UNA ESTRUCTURA

Como habíamos mencionado anteriormente una estructura puede contener como campo a cualquier tipo de variable permitido. Si el usuario definió una estructura anteriormente se puede utilizar a ésta como campo de otra estructura.

En el caso de que no haya sido definida anteriormente puede definírsela dentro de la estructura en la que se la quiere utilizar.

Solo hay que tener en cuenta que para referenciar al campo de la estructura que es un campo de la misma, se debe utilizar el nombre de la primera luego el operador punto, el nombre del campo que es una estructura luego el operador punto y por último el nombre del campo al que se hace referencia en la misma.

Siempre que usemos una estructura para referenciar un campo se deberá utilizar el operador punto (.). Si se utiliza mas de una estructura dentro de otra, se debe utilizar el punto por cada nivel que se desee profundizar.

Cuando una estructura tiene como campo a otra, decimos que están anidadas. Al igual que cuando se anidan lazos repetitivos o condicionales en un programa.

Ejemplo

```
#include<stdio.h>
#include<conio.h>
#include<string.h>

/* Se definen la cantidad de empleados y el largo de los strings */

#define    MAX    20
#define    LARGO    30

/* Se definen las estructuras que formarán los datos de los      *
 * empleados                                                    */

struct dat_pers { char nomb[LARGO],apell[LARGO];};
struct fecha    { int anio,mes,dia;};
struct domic    { char dire[LARGO],local[LARGO],prov[LARGO];};
struct tele     { unsigned int carac,numer;};

/* Estructura para guardar los datos de los empleados */

struct agend     { struct dat_pers datos;
                  struct domic direccion;
                  struct fecha cumple,antig;
                  unsigned int leg;
                  struct tele partic,movil;};

void main(void)
{
    /* Se declaran las variables */
    struct agend empleados[MAX];
    char i,resp,enc;
    unsigned int cons;
```

```

/* Ingreso de los datos */

for(i=0;i<MAX;i++)
{
    clrscr();
    fflush(stdin);
    printf("\t\tIngrese los datos del personal\n\n");
    printf("\nApellido : ");
    gets(empleados[i].datos.apell);
    printf("\nNombre : ");
    gets(empleados[i].datos.nomb);
    printf("\nDirección : ");
    gets(empleados[i].direccion.dire);
    printf("\nLocalidad : ");
    gets(empleados[i].direccion.local);
    printf("\nProvincia : ");
    gets(empleados[i].direccion.prov);
    printf("\nFecha de nacimiento (día/mes/año) : ");
    scanf("%d/%d/%d",&empleados[i].cumple.dia, \
    &empleados[i].cumple.mes,&empleados[i].cumple.anio);
    printf("\nFecha de ingreso ( día/mes/año) : ");
    scanf("%d/%d/%d",&empleados[i].antig.dia, \
    &empleados[i].antig.mes,&empleados[i].antig.anio);
    printf("\nTeléfono particular (característica-número) : ");
    scanf("%u-%u",&empleados[i].partic.carac,\
    &empleados[i].partic.numer);
    printf("\nTeléfono móvil (característica-número) : ");
    scanf("%u-%u",&empleados[i].movil.carac,\
    &empleados[i].movil.numer);
    printf("\nNúmero de legajo : ");
    scanf("%u",&empleados[i].leg);
}

clrscr();
resp='S';

/* Consulta de un empleado */

while(resp=='S' || resp=='s')
{
    enc=0;
    clrscr();
    printf("\n\nIngrese el número de legajo : ");
    scanf("%u",&cons);

    /* Búsqueda del empleado */

    for(i=0;i<MAX&& !enc;i++)
        if(cons==empleados[i].leg)
        {
            enc=1;
            i--;
        }
}

```

```

/* Impresión de los datos del empleado */

if(enc)
{
    printf("El empleado %s,%s\n",\
    empleados[i].datos.apell,empleados[i].datos.nomb);
    printf("se domicilia en %s,%s,%s\n",\
    empleados[i].direccion.dire,\
    empleados[i].direccion.local,\
    empleados[i].direccion.prov);
    printf("el teléfono particular es : %u-%u \t",\
    ,empleados[i].partic.carac);
    printf("el celular es 15-%u-%u",\
    empleados[i].movil.carac,\
    empleados[i].movil.numer);
    printf("\nel ingreso en la empresa fue el);
    printf("%d/%d/%d,",empleados[i].antig.dia,\
    empleados[i].antig.mes, empleados[i].antig.anio);
    printf("su retiro se producirá en el año %d ",\
    empleados[i].cumple.anio+65);
}
else
    printf("Este legajo no existe");

/* Nueva consulta */

printf("\n\nDesea buscar otro empleado (S/N)? ");
resp=getche();
}
}

```

Este programa permite el ingreso de los datos de los empleados de una empresa para su consulta mediante el número del legajo.

En él se definen cuatro estructuras que luego son utilizadas como campos de la estructura en donde se guardan los datos de los empleados.

La estructura "dat_pers" contiene dos campos los cuales se utilizan para guardar el nombre y el apellido de la persona.

La denominada fecha sirve para guardar los datos de las fechas de nacimiento como la de ingreso a la empresa.

En la que se llama "domic" se ubican los datos del domicilio localidad y provincia.

En "tele" se utiliza para almacenar el número de teléfono de la forma de característica y número.

Todas ellas constituyen la estructura "agend" que será la encargada de guardar los datos de estos empleados.

Si bien la forma de acceder a cada campo de la estructura del tipo "agend" llamada "empleados" es complicada tiene la ventaja de poder empaquetar los datos y, además, es posible cambiar datos complejos en dos posiciones del vector solamente con asignar un campo a otro siempre que ambos sean del mismo tipo.

PASAJE Y RETORNO DE MIEMBROS DE UNA ESTRUCTURA A UNA FUNCIÓN

Un miembro o campo de una estructura es en sí mismo una variable de un tipo especificado, y como toda variable se puede enviar o retornar desde una función siguiendo las reglas propias de las funciones para recibir parámetros, o retornar un valor desde la misma.

Ejemplo

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
/* Se define el valor de  $\pi$  */
#define PI 3.14159
/* Definición de la estructura */
struct complejo { float x,y;};
/* Prototipo de las funciones */
float modulo(float,float);
float argumento(float,float);
float absi(float,float);
float orde(float,float);
void main(void)
{
    /* Declaración de las variables */
    char c;
    struct complejo A,B;
    do
    {
        clrscr();
        printf("\n\n\n\t\t\tPasaje de coordenadas");
        printf("\n\n\t\t\tIngrese la opción");
        printf("\n\n\t\t\t(P) polar a cartesiana");
        printf("\n\n\t\t\t(C) cartesiana a polar ");
        printf("\n\n\t\t\t(S) salir          ");
        c=getche();
        switch(c)
        {
            /* Elección de pasaje de polar a cartesiana */
            case 'c': case 'C': printf("\n\nIngrese el modulo : ");
                                scanf("%f",&A.x);
                                printf("\nIngrese el argumento : ");
                                scanf("%f",&A.y);
                                B.x=absi(A.x,A.y);
                                B.y=orde(A.x,A.y);
                                printf("\n\nLa abscisa es : %f",B.x);
                                printf("\n\nLa ordenada es : \n\t\t\t\t\t%f",B.y);
                                printf("\n\nPresione una tecla ");
                                printf("para continuar");
                                getch();
                                break;
```

```

        /* Elección de pasaje de cartesiana a polar */
        case 'P': case 'p': printf("\n\nIngrese la abscisa : ");
                            scanf("%f",&A.x);
                            printf("\nIngrese la ordenada : ");
                            scanf("%f",&A.y);
                            B.x=modulo(A.x,A.y);
                            B.y=argumento(A.x,A.y);
                            printf("\n\nEl modulo es : %f",B.x);
                            printf("\n\nEl argumento es : \
                                %f",B.y);
                            printf("\n\nPresione una tecla ");
                            printf("para continuar");
                            getch();
                            break;
        /* Opción de salir */
        case 'S': case 's': c=0;
    }
}
while(c);
}
/* Cálculo de la abscisa */
float absi(float r,float f)
{
    float ab;
    ab=r*cos(f*PI/180);
    return (ab);
}
/* cálculo de la ordenada */
float orde(float r,float f)
{
    float or;
    or=r*sin(f*PI/180);
    return (or);
}
/* cálculo del modulo */
float modulo(float x,float y)
{
    float mo;
    mo=x*x+y*y;
    mo=sqrt(mo);
    return (mo);
}
/* cálculo del argumento */
float argumento(float x,float y)
{
    float ar;
    ar=atan2(y,x);
    ar*=(180/PI);
    return (ar);
}

```

En el ejemplo mostrado, se utiliza para pasar coordenadas de un punto de forma cartesiana a polar y viceversa.

En él se puede observar que se definen cuatro funciones para realizar dicha tarea. Estas reciben como parámetros dichas coordenadas y devuelven un único valor. Los mismos provienen de campos de una estructura y son devueltos a un campo de otra.

PASAJE DE ESTRUCTURAS A FUNCIONES

Cuando se desea pasar un valor de una estructura a una función, esto se puede realizar utilizando el nombre de la estructura como parámetro cuando se invoca a la función.

Para ello es necesario declarar a la estructura como tipo global, es decir, definirla antes del comienzo del programa y afuera de todas las funciones.

En el prototipo de la función se debe declarar el tipo del parámetro utilizando la palabra reservada "struct" y a continuación el nombre del tipo de la estructura.

Cuando se define a la función se debe agregar al prototipo de la misma el nombre de la variable que va a corresponder al tipo de la estructura correspondiente.

Únicamente se pueden recibir estructuras como parámetros siempre que coincidan los tipos de estructuras que se reciben en la función y los tipos que se envían cuando se la invoca.

Ejemplo

```
#include<stdio.h>
#include<conio.h>
#include<math.h>

/* Se define el valor de p */
#define PI 3.14159

/* Definición de la estructura */
struct complejo { float x,y;};

/* Prototipo de las funciones */
float modulo(struct complejo);
float argumento(struct complejo);
float absi(struct complejo);
float orde(struct complejo);

void main(void)
{
    /* declaración de las variables */
    char c;
    struct complejo A,B;
    do
    {
        clrscr();
        printf("\n\n\n\t\t\tPasaje de coordenadas");
        printf("\n\n\t\t\tIngrese la opción");
        printf("\n\n\t\t\t(P) polar a cartesiana");
        printf("\n\n\t\t\t(C) cartesiana a polar ");
        printf("\n\n\t\t\t(S) salir          ");
        c=getche();
    }
```

```

switch(c)
{
    /* Elección de pasaje de polar a cartesiana */
    case 'c': case 'C': printf("\n\nIngrese el modulo : ");
                        scanf("%f",&A.x);
                        printf("\n\nIngrese el argumento : ");
                        scanf("%f",&A.y);
                        B.x=absi(A);
                        B.y=orde(A);
                        printf("\n\nLa abscisa es : %f",B.x);
                        printf("\n\nLa ordenada es : \
                            %f",B.y);
                        printf("\n\nPresione una tecla ");
                        printf("para continuar");
                        getch();
                        break;

    /* Elección de pasaje de cartesiana a polar */
    case 'P': case 'p': printf("\n\nIngrese la abscisa : ");
                        scanf("%f",&A.x);
                        printf("\n\nIngrese la ordenada : ");
                        scanf("%f",&A.y);
                        B.x=modulo(A);
                        B.y=argumento(A);
                        printf("\n\nEl modulo es : %f",B.x);
                        printf("\n\nEl argumento es : \
                            %f",B.y);
                        printf("\n\nPresione una tecla ");
                        printf("para continuar");
                        getch();
                        break;

    /* opción de salir */
    case 'S': case 's': c=0;

}
}
while(c);
}
/* cálculo de la abscisa */
float absi(struct complejo A)
{
    float ab;
    ab=A.x*cos(A.y*PI/180);
    return (ab);
}
/* cálculo de la ordenada */
float orde(struct complejo A)
{
    float or;
    or=A.x*sin(A.y*PI/180);
    return (or);
}
/* cálculo del modulo */
float modulo(struct complejo A)
{
    float mo;
    mo=A.x*A.x+A.y*A.y;
    mo=sqrt(mo);
    return (mo);
}

```



```

/* cálculo del argumento */
float argumento(struct complejo A)
{
    float ar;
    ar=atan2(A.y,A.x);
    ar*=(180/PI);
    return (ar);
}

```

En este ejemplo encontramos una modificación a la que propusimos en el párrafo anterior enviando como parámetro de las funciones la estructura llamada "A". Esto nos evita enviar dos parámetros a cada función, como si no estuviesen relacionados entre sí. Se observa que las funciones tratan a cada uno de los campos de la estructura que fue enviada de la misma forma como si se hubieran enviado los parámetros por separado.

RETORNO DEL VALOR DE UNA ESTRUCTURA DESDE UNA FUNCIÓN

Para retornar el valor de una estructura desde una función solo hay que utilizar la palabra reservada "return" y entre paréntesis el nombre de la estructura deseada.

En el prototipo de la función hay que declarar el tipo de la estructura como valor devuelto por la función. Esto se realiza utilizando la palabra reservada "struct" y a continuación el nombre del tipo de la estructura y a continuación el nombre de la función.

Cuando se la define se procede de la forma citada anteriormente.

Solo es posible recibir el valor de una estructura si la variable que la va a recibir es del mismo tipo que el que retorna la función.

Ejemplo

```

#include<stdio.h>
#include<conio.h>
#include<math.h>
#include<ctype.h>

/* Se define una estructura para guardar un número complejo *
 * el campo x puede ser la parte real o el modulo,          *
 * el campo y puede ser la parte imaginaria o el argumento, *
 * el campo t es la forma en que está expresado el complejo */
struct complejo { float x,y;
                  char t;};

/* Se declaran los prototipos de las funciones a utilizar */
void sumar(void);
void restar(void);
void producto(void);
void cociente(void);
struct complejo ingreso(void);
struct complejo convacart(struct complejo);
struct complejo convapol(struct complejo);
void imprime(struct complejo,struct complejo,struct complejo);

```

```

/* Comienzo de la función main */
void main(void)
{
    char c;
    do
    {
        /* Menú de opciones para elegir el cálculo */
        clrscr();
        printf("\n\n\n\t\t\tCálculos complejos");
        printf("\n\n\t\t\tIngrese una opción");
        printf("\n\n\t\t\t (S) Suma");
        printf("\n\n\t\t\t (R) Resta");
        printf("\n\n\t\t\t (P) Producto");
        printf("\n\n\t\t\t (C) Cociente");
        printf("\n\n\t\t\t (Q) Salir\t");
        c=getche();
        switch(c)
        {
            case 's': case 'S': sumar();
                                break;
            case 'r': case 'R': restar();
                                break;
            case 'p': case 'P': producto();
                                break;
            case 'c': case 'C': cociente();
                                break;
        }
    }
    while((c!='q')&&(c!='Q'));
}

/* Función que realiza la suma de dos números complejos */
void sumar(void)
{
    /* Se declaran las variables en donde se almacenan *
     * los números complejos */
    struct complejo A,B,C;
    /* Se invoca la función que permite el ingreso de *
     * los datos */
    A=ingreso();
    B=ingreso();
    clrscr();
    printf("\n\n");
    /* Se imprimen los datos de los operandos en la *
     * notación en que fueron ingresados */
    if(A.t=='P')
        printf("%f e^%f +",A.x,A.y);
    else
        printf("%f j%f +",A.x,A.y);
    if(B.t=='P')
        printf(" %f e^%f = ",B.x,B.y);
    else
        printf(" %f j%f = ",B.x,B.y);
    /* Cambio del tipo de notación de cada operando *
     * si se hubiese ingresado en forma polar */
    if(A.t=='P')
        A=convacart(A);
}

```

```

        if(B.t=='P')
            B=convacart(B);
        C.x=A.x+B.x;
        C.y=A.y+B.y;
        C.t='C';
        /* Se invoca a la función de impresión de los resultados */
        imprime(C,A,B);
    }

/* Función que realiza la resta de dos números complejos */
void restar(void)
{
    /* Se declaran las variables en donde se almacenan *
     * los números complejos */
    struct complejo A,B,C;
    /* Se invoca la función que permite el ingreso de *
     * los datos */
    A=ingreso();
    B=ingreso();
    clrscr();
    printf("\n\n");
    /* Se imprimen los datos de los operandos en la *
     * notación en que fueron ingresados */
    if(A.t=='P')
        printf("%f e^%f -",A.x,A.y);
    else
        printf("%f j%f -",A.x,A.y);
    if(B.t=='P')
        printf(" %f e^%f = ",B.x,B.y);
    else
        printf(" %f j%f = ",B.x,B.y);
    /* Cambio del tipo de notación de cada operando *
     * si se hubiese ingresado en forma polar */
    if(A.t=='P')
        A=convacart(A);
    if(B.t=='P')
        B=convacart(B);
    C.x=A.x-B.x;
    C.y=A.y-B.y;
    C.t='C';
    /* Se invoca a la función de impresión de los resultados */
    imprime(C,A,B);
}

/* Función que realiza el producto de dos números complejos */
void producto(void)
{
    /* Se declaran las variables en donde se almacenan *
     * los números complejos */
    struct complejo A,B,C;
    /* Se invoca la función que permite el ingreso de *
     * los datos */
    A=ingreso();
    B=ingreso();
    clrscr();
    printf("\n\n");

```

```

/* Se imprimen los datos de los operandos en la *
 * notación en que fueron ingresados */
if(A.t=='P')
    printf("%f e^%f *",A.x,A.y);
else
    printf("%f j%f *",A.x,A.y);
if(B.t=='P')
    printf(" %f e^%f = ",B.x,B.y);
else
    printf(" %f j%f = ",B.x,B.y);
/* Cambio del tipo de notación de cada operando *
 * si se hubiese ingresado en forma cartesiana */
if(A.t=='C')
    A=convapol(A);
if(B.t=='C')
    B=convapol(B);
C.x=A.x*B.x;
C.y=A.y+B.y;
C.t='P';
/* Se invoca a la función de impresión de los resultados */
imprime(C,A,B);
}

/* Función que realiza el cociente de dos números complejos */
void cociente(void)
{
    /* Se declaran las variables en donde se almacenan *
     * los números complejos */
    struct complejo A,B,C;
    /* Se invoca la función que permite el ingreso de *
     * los datos */
    A=ingreso();
    B=ingreso();
    clrscr();
    printf("\n\n");
    /* Se imprimen los datos de los operandos en la *
     * notación en que fueron ingresados */
    if(A.t=='P')
        printf("%f e^%f /",A.x,A.y);
    else
        printf("%f j%f /",A.x,A.y);
    if(B.t=='P')
        printf(" %f e^%f = ",B.x,B.y);
    else
        printf(" %f j%f = ",B.x,B.y);
    /* Cambio del tipo de notación de cada operando *
     * si se hubiese ingresado en forma cartesiana */
    if(A.t=='C')
        A=convapol(A);
    if(B.t=='C')
        B=convapol(B);
    C.x=A.x/B.x;
    C.y=A.y-B.y;
    C.t='P';
    /* Se invoca a la función de impresión de los resultados */
    imprime(C,A,B);
}

```

```

/* Función que permite el ingreso de un número complejo *
 * esta retorna el valor de la estructura, la misma *
 * contiene la parte real e imaginaria o el modulo y el *
 * argumento y, además, el tipo de la notación en que *
 * esta escrito */
struct complejo ingreso(void)
{
    /* Declara la variable en donde se va a almacenar *
     * el número complejo */
    struct complejo A;
    /* Variable auxiliar para evitar el problema de la *
     * función "scanf" */
    float aux;
    char c;
    do
    {
        /* Menú para seleccionar la forma en que *
         * se ingresa el número */
        clrscr();
        printf("\n\n\n\t\t\tIngreso del Valor");
        printf("\n\n\t\t\tIngrese la opción");
        printf("\n\n\t\t\t(P) Notación polar");
        printf("\n\n\t\t\t(C) Notación cartesiana\t");
        c=getche();
        switch(c)
        {
            /* Ingreso en notación polar */
            case 'p': case 'P': printf("\n\nModulo : ");
                                scanf("%f",&aux);
                                A.x=aux;
                                printf("\nArgumento : ");
                                scanf("%f",&aux);
                                A.y=aux;
                                A.t=toupper(c);
                                c=0;
                                break;

            /* Ingreso en notación cartesiana */
            case 'c': case 'C': printf("\n\nParte real : ");
                                scanf("%f",&aux);
                                A.x=aux;
                                printf("\nParte imaginaria : ");
                                scanf("%f",&aux);
                                A.y=aux;
                                A.t=toupper(c);
                                c=0;
                                break;

        }
    }
    while(c);
    /* Aquí se retorna la variable de tipo "struct complejo */
    return(A);
}

```

```

/* Función que convierte un número en notación polar a *
 * notación cartesiana */
struct complejo convacart(struct complejo A)
{
    /* Variable en donde se guardara el cambio de notación */
    struct complejo B;
    B.x=A.x*cos(A.y*M_PI/180);
    B.y=A.x*sin(A.y*M_PI/180);
    B.t=A.t;
    /* Aquí se retorna la variable de tipo "struct complejo */
    return(B);
}

/* Función que convierte un número en notación cartesiana a *
 * notación polar */
struct complejo convapol(struct complejo A)
{
    /* Variable en donde se guardara el cambio de notación */
    struct complejo B;
    B.x=sqrt((A.x*A.x)+(A.y*A.y));
    B.y=atan2(A.y,A.x)*(180/M_PI);
    B.t=A.t;
    /* Aquí se retorna la variable de tipo "struct complejo */
    return(B);
}

/* Función que imprime el resultado del cálculo realizado */
void imprime(struct complejo C,struct complejo A,struct complejo B)
{
    /* Verifica la notación en que se ingresaron a los operandos *
     * Si ambos están expresados en la misma forma el resultado *
     * se expresa en esa notación, en caso contrario en la *
     * notación natural del cálculo realizado */
    if(A.t==B.t)
    {
        switch(A.t)
        {
            case 'P': if(C.t=='C')
                        C=convapol(C);
                        printf(" %f e^%f ",C.x,C.y);
                        break;
            case 'C': if(C.t=='P')
                        C=convacart(C);
                        printf(" %f j%f ",C.x,C.y);
                        break;
        }
    }
    else
    {
        if(C.t=='p')
            printf(" %f e^%f ",C.x,C.y);
        else
            printf(" %f j%f ",C.x,C.y);
        printf("\n\nPresione una tecla para continuar ");
        getch();
    }
}

```

En este ejemplo se desarrolla un programa que permite realizar las cuatro operaciones aritméticas entre dos números complejos.

Se definen funciones para realizar cada uno de los cálculos en forma independiente. Todas ellas invocan a dos funciones, la primera permite el ingreso de los valores para realizar los cálculos y la segunda imprime el valor del mismo. En la primera se devuelve el valor ingresado en forma de una estructura.

Existen dos funciones que sirven para realizar la conversión del tipo de notación en que ha sido ingresado el valor a la notación para realizar el cálculo de la forma más simple. Esta devuelve una estructura y también recibe como parámetro el mismo tipo de estructura.

TAMAÑO Y UBICACIÓN DE LOS CAMPOS EN LA MEMORIA R.A.M

El tamaño de una estructura está dado por la cantidad y tipos de campos que la integran. Es decir, como cada campo es en sí mismo una variable, ésta ocupa el lugar definido por su tipo asociado. Para calcular la medida de una estructura solo hay que sumar las magnitudes individuales de cada uno de sus campos.

Podemos calcular el tamaño de la siguiente estructura:

```
struct lista {  int orden;
                char nombre[30], apellido[20];
                float notas[10,6], promedio[10]};
```

En la estructura antes definida podemos observar que posee 5 campos, uno de tipo int, dos de tipo char y dos de tipo float. El primer campo es de tipo int por lo tanto ocupa 2 bytes en el sistema operativo D.O.S., el segundo y tercer campo son de tipo char (1 byte), pero como son vectores cada uno ocupa la dimensión del tipo multiplicado por la cantidad de posiciones, en el campo "nombre" ocupa 30 posiciones por 1 byte cada una dando un total de 30 bytes para este campo; el tercero cuyo nombre es "apellido" tiene 20 posiciones, lo cual da una medida de 20 bytes.

Los dos últimos son de tipo float (4 bytes), el campo "notas" es una matriz de 10 por 6, éste tiene una cantidad total de posiciones igual a 60, dando un total para éste de 240 bytes. Por último el campo "promedio" es un vector con 10 posiciones de largo, dando un valor total de 40 bytes.

El tamaño total de la estructura "lista" es de:

2 bytes +30 bytes +20 bytes +240 bytes +40 bytes =332 bytes

La estructura se ubica en una posición memoria R.A.M. determinada por el sistema operativo, para D.O.S., se encuentra en el segmento de datos de la misma. En ella comienzan a ubicarse los campos uno a continuación del otro según se halla definido en su declaración.

Por ejemplo si definimos la siguiente:

```
struct fecha {    int anio;  
                char mes,dia} fecha_de_hoy;
```

fecha de hoy	anio		A390
			A391
	mes		A392
	dia		A393

Como podemos observar la estructura "fecha_de_hoy" ocupa 4 bytes y se ubican los campos en el siguiente orden: primero "anio", segundo "mes" y por último "dia".

UNIONES

Una unión permite compartir la misma posición de memoria para varias variables. Estas pueden ser del mismo tipo o de tipos diferentes.

Cuando son de distinto tipo, obviamente no tienen el mismo largo, y la de mayor tamaño es la que define el espacio de memoria que ocupa la unión.

A cada variable que compone la unión se la denomina "campo" (al igual que las estructuras).

Cada campo comienza en la misma posición de memoria y ocupa el largo correspondiente al tipo de la misma.

No se debe confundir una estructura con una unión, si bien ambas son parecidas, hay una gran diferencia entre ellas. Las primeras ubican a sus campos uno a continuación del otro mientras que en las uniones los campos comparten la misma ubicación en la memoria de la computadora, todos comienzan en la misma dirección de memoria.

Su importancia radica en que se pueden ingresar datos utilizando un tipo de variable y extraerlos utilizando otro, otra de sus ventajas es que cuando uno posee una cantidad limitada de memoria para datos se puede utilizar la misma ubicación de memoria para guardar datos de distinto tipo, con el cuidado de no superponer diferentes variables al mismo tiempo.

DECLARACIÓN

La declaración de una unión se puede realizar de tres diferentes formas.

```
union etiqueta {tipo campo_1;  
                tipo campo_2;  
                tipo campo_3;  
                ...  
                ...  
                tipo campo_n;} lista_de_variables;
```



```

union { tipo campo_1;
        tipo campo_2;
        tipo campo_3;
        ...
        ...
        tipo campo_n;} lista_de_variables;

```

```

union etiqueta {tipo campo_1;
                tipo campo_2;
                tipo campo_3;
                ...
                ...
                tipo campo_n;};

```

La primera es la más general, se debe utilizar la palabra reservada "unión", luego un nombre del tipo "etiqueta", a continuación las llaves de apertura y de cierre ({ }), que se utilizan para encerrar los campos, y las variables que tendrán el tipo de la unión definido, en la lista_de_variables.

Esta forma se la utiliza para declarar uniones en forma global o local para cada función. En la segunda forma se omite la etiqueta, pero aquí debe incluirse la lista_de_variables. En el caso de que no se especifique la lista no se podrá utilizar, debido a que no es posible realizar una identificación de ésta.

La última consiste en omitir la lista_de_variables, al igual que en las estructuras, ésta es la más utilizada pudiéndose utilizar en cualquier función o también definirla de forma global, para ello solo basta con declarar el tipo en el momento en que se precise. Para declarar una o más variables se debe realizar lo siguiente:

```

union etiqueta lista_de_variables;

```

Ejemplo

```

#include<stdio.h>
union data { int valor;
             char dato[2];}ent[100];
void ingreso(void);
void main(void)
{
    union data salida;
    -----
    -----
    -----
}
void ingreso(void)
{
    union data entrada;
    -----
    -----
    -----
}

```

Podemos observar que en el ejemplo se declaró una unión denominada data, ésta contiene dos campos uno de tipo "int" y el otro es un vector de dos posiciones de tipo "char".

Se declara un vector de 100 posiciones llamado "ent" como variable global, que es visto por las funciones "main" e "ingreso".

En cada una de las funciones se declara una variable del tipo "union data" que solo es vista por la función en la que fue declarada.

```
#include <stdio.h>
union {int A;
      float B;} calc;
void soluc(void);
void main(void)
{
    union {int A;
          float B;} calc;
    -----
    -----
    -----
}
void soluc(void)
{
    union {int A;
          float B;} clave;
    -----
    -----
    -----
}
```

Aquí podemos observar que las uniones definidas en este ejemplo no tienen la etiqueta y por lo tanto no se puede reutilizar este tipo con otras variables. Si se quiere usar un tipo de unión igual al definido en forma global hay que redefinirlo. Pero realizando esto los tipos de las uniones no son idénticos y no es posible asignar una variable a otra.

También se puede observar que se han definido en cada función uniones iguales en forma local a ellas.

```
#include <stdio.h>
union un1{ char A;
          unsigned long B;};
void disposit(void);
void main(void)
{
    union un1 clase;
    -----
    -----
    -----
}
void disposit(void)
{
    union un1 sensado;
    -----
    -----
    -----
}
```

Aquí tenemos una definición de un tipo global de unión denominado "un1", este es global y por lo tanto este tipo es visto por todas las funciones del programa. Como tiene una etiqueta que identifica al tipo de la unión se pueden declarar variables de tipo local o global solamente utilizando la siguiente sintaxis : union un1 seguido por el o los nombres de las variables que se deseen declarar bajo ese tipo.

REFERENCIA Y ASIGNACIÓN A CAMPOS EN UNA UNIÓN

Para hacer referencia a un campo de una unión, se utiliza el operador punto o miembro, invocando primero el nombre de la unión, seguido del operador punto y por último el nombre del campo, de forma similar a la que se utiliza en una estructura.

Cuando se desea realizar una asignación a un campo en la unión se utiliza el signo igual (=), al ejecutar esto último solo debe tenerse en cuenta que las variables a asignar deben ser necesariamente del mismo tipo.

Se puede asignar una unión a otra, pero ambas deben ser del mismo tipo, ambas deben tener la misma declaración, o etiqueta.

Ejemplo

```
#include<stdio.h>
#include<conio.h>
/* Se define una unión */
union alfa { int A[2];
             char B[4];
             float C;};

void main(void)
{
    /* Se declara una variable del tipo "union alfa" */
    union alfa dato;
    /* Asigno e imprimo valores al campo "A" */
    dato.A[0]=6529;
    dato.A[1]=30287;
    clrscr();
    printf("\n\ndato.A[0]=%d\tdato.A[1]=%d",dato.A[0],dato.A[1]);
    printf("\nEn formato hexadecimal");
    printf("\ndato.A[0]=%X\tdato.A[1]=%X",dato.A[0],dato.A[1]);
    /* Asigno e imprimo valores al campo "B" */
    dato.B[0]='A';
    dato.B[1]='B';
    dato.B[2]='C';
    dato.B[3]='D';
    printf("\n\ndato.B[0]=%c\tdato.B[1]=%c",dato.B[0],dato.B[1]);
    printf("\tdato.B[2]=%c\tdato.B[3]=%c".dato.B[2],dato.B[3]);
    printf("\nEn formato ASCII");
    printf("\ndato.B[0]=%d\tdato.B[1]=%d\tdato.B[2]=%d\t\
        dato.B[3]=%d",dato.B[0],dato.B[1],dato.B[2],dato.B[3]);
    printf("\nEn formato hexadecimal");
    printf("\ndato.B[0]=%X\tdato.B[1]=%X\tdato.B[2]=%X\t\
        dato.B[3]=%X",dato.B[0],dato.B[1],dato.B[2],dato.B[3]);
}
```

```

/* Asigno e imprimo un valor al campo "C" */
dato.C=3.123e-10;
printf("\n\ndato.C=%G",dato.C);
printf("\n\nPresione una tecla para continuar");
getch();
/* Asigno e imprimo valores al campo "A" */
dato.A[0]=6529;
dato.A[1]=30287;
clrscr();
printf("\n\ndato.A[0]=%d\tdato.A[1]=%d",dato.A[0],dato.A[1]);
printf("\nEn formato hexadecimal");
printf("\ndato.A[0]=%X\tdato.A[1]=%X",dato.A[0],dato.A[1]);
/* visualizo los valores en el campo "B" */
printf("\n\nSin cambiar los valores examinamos el campo ""B""");
printf("\n\ndato.B[0]=%c\tdato.B[1]=%c",dato.B[0],dato.B[1]);
printf("\tdato.B[2]=%c\tdato.B[3]=%c",dato.B[2],dato.B[3]);
printf("\nEn formato ASCII");
printf("\ndato.B[0]=%d\tdato.B[1]=%d\tdato.B[2]=%d\t\tdato.B[3]=%d",dato.B[0],dato.B[1],dato.B[2],dato.B[3]);
printf("\nEn formato hexadecimal");
printf("\ndato.B[0]=%X\tdato.B[1]=%X\tdato.B[2]=%X\t\tdato.B[3]=%X",dato.B[0],dato.B[1],dato.B[2],dato.B[3]);
/* visualizo los valores en el campo "C" */
printf("\n\nSin cambiar los valores examinamos el campo ""C""");
printf("\n\ndato.C=%G",dato.C);
printf("\n\nPresione una tecla para continuar");
getch();
clrscr();
/* Asigno e imprimo valores al campo "B" */
dato.B[0]='A';
dato.B[1]='B';
dato.B[2]='C';
dato.B[3]='D';
printf("\n\ndato.B[0]=%c\tdato.B[1]=%c",dato.B[0],dato.B[1]);
printf("\tdato.B[2]=%c\tdato.B[3]=%c",dato.B[2],dato.B[3]);
printf("\nEn formato ASCII");
printf("\ndato.B[0]=%d\tdato.B[1]=%d\tdato.B[2]=%d\t\tdato.B[3]=%d",dato.B[0],dato.B[1],dato.B[2],dato.B[3]);
printf("\nEn formato hexadecimal");
printf("\ndato.B[0]=%X\tdato.B[1]=%X\tdato.B[2]=%X\t\tdato.B[3]=%X",dato.B[0],dato.B[1],dato.B[2],dato.B[3]);
/* visualizo los valores en el campo "A" */
printf("\n\nSin cambiar los valores examinamos el campo ""A""");
printf("\n\ndato.A[0]=%d\tdato.A[1]=%d",dato.A[0],dato.A[1]);
printf("\nEn formato hexadecimal");
printf("\ndato.A[0]=%X\tdato.A[1]=%X",dato.A[0],dato.A[1]);
/* visualizo los valores en el campo "C" */
printf("\n\nSin cambiar los valores examinamos el campo ""C""");
printf("\n\ndato.C=%G",dato.C);
printf("\n\nPresione una tecla para continuar");
getch();
clrscr();
/* Asigno e imprimo un valor al campo "C" */
dato.C=3.123e-10;
printf("\n\ndato.C=%G",dato.C);
/* visualizo los valores en el campo "A" */
printf("\n\nSin cambiar los valores examinamos el campo ""A""");
printf("\n\ndato.A[0]=%d\tdato.A[1]=%d",dato.A[0],dato.A[1]);

```

```

printf("\nEn formato hexadecimal");
printf("\ndato.A[0]=%X\tdato.A[1]=%X", dato.A[0], dato.A[1]);
/* visualizo los valores en el campo "B" */
printf("\n\nSin cambiar los valores examinamos el campo ""B""");
printf("\ndato.B[0]=%c\tdato.B[1]=%c", dato.B[0], dato.B[1]);
printf("\tdato.B[2]=%c\tdato.B[3]=%c", dato.B[2], dato.B[3]);
printf("\nEn formato ASCII");
printf("\ndato.B[0]=%d\tdato.B[1]=%d\tdato.B[2]=%d\t\
      dato.B[3]=%d", dato.B[0], dato.B[1], dato.B[2], dato.B[3]);
printf("\nEn formato hexadecimal");
printf("\ndato.B[0]=%X\tdato.B[1]=%X\tdato.B[2]=%X\t\
      dato.B[3]=%X", dato.B[0], dato.B[1], dato.B[2], dato.B[3]);
printf("\n\nPresione una tecla para terminar");
getch();
}

```

En este ejemplo se muestra como se puede asignar y referirse a un campo de la unión. Se asignan valores por separado a cada campo y luego se los imprime.

Seguidamente se asignan valores a un campo y luego se imprimen todos los campos. Este procedimiento permite ver como se comparten las posiciones de memoria en una unión, pudiendo visualizar que si se utiliza erróneamente un campo se puede perder la información contenida en otro.

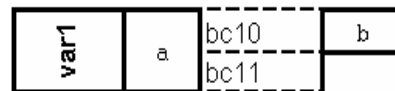
UBICACIÓN Y TAMAÑO DE LOS CAMPOS DE UNA UNIÓN EN LA MEMORIA

Una unión se ubica en la zona de memoria perteneciente al segmento de datos, a partir de la dirección donde comienza la unión se ubican los campos, pero todos comienzan en ese preciso lugar, pero el largo de la unión, lo determina el campo de mayor tamaño. Por ejemplo en la siguiente unión

```

union ej1 { int a;
            char b;} var1;

```



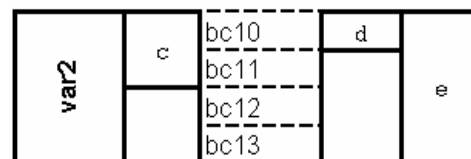
Podemos observar que esta compuesta por dos campos uno de tipo int (en D.O.S. 2 bytes) y el otro de tipo char (en D.O.S. 1 byte). Ambos se ubicaran a partir de la misma dirección de memoria, manteniendo su dimensión típica, es decir, el campo int ocupara dos bytes mientras que el char ocupa solamente uno, dando un tamaño total para la unión de 2 bytes.

Si analizamos la unión que sigue a continuación

```

union ej2 { int c;
            char d;
            float e;} var2;

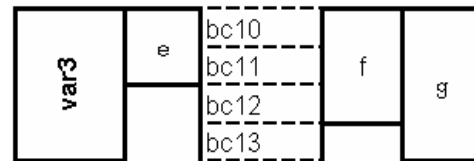
```



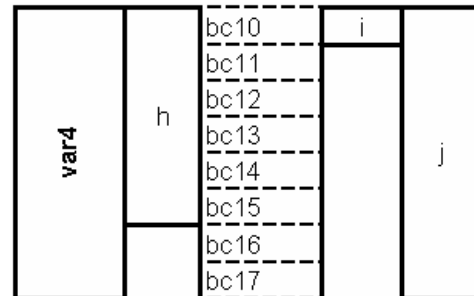
Encontraremos que los tres campos también comienzan en la misma dirección pero como en este caso tenemos además un float (en D.O.S. 4 bytes) cambiará solamente el largo de la misma siendo en este caso de 4 bytes, debido a que el tipo de mayor tamaño es el float.

Cuando nos encontramos en presencia de un vector este comenzará en la dirección de comienzo de la unión, ocupando los lugares que corresponden a su largo. Esto no significa que determine el largo de la unión, lo determinará el campo de mayor tamaño. Veamos los siguientes ejemplos:

```
union ej3 { int e;  
            char f[3];  
            float g;} var3;
```



```
union ej4 { int h[3];  
            char i;  
            float j[2];} var4;
```



En el primero nos encontramos con un vector de char con un largo de 3 posiciones, una variable entera y una de punto flotante, como la de mayor tamaño de los tres campos es el float éste determinará el largo total de la estructura.

En el segundo nos encontramos con dos vectores uno int de 3 posiciones (6 bytes) y otro de tipo float de 2 posiciones (8 bytes), y además una variable de tipo char (1 byte), aquí encontraremos que el vector de punto flotante es el de mayor largo, dando como resultado que este tamaño será el de la unión.

PASAJE Y RETORNO DE MIEMBROS DE UNA UNIÓN A UNA FUNCIÓN

Los campos de una unión son en sí mismos variables como cualquier otra, tienen un tipo asociado y un identificador válido (nombre) esto hace que se pueda retornar un campo de una unión desde una función o se pueda enviar el mismo como parámetro de la misma.

Ejemplo

```
#include<stdio.h>
#include<conio.h>
/* Se define el tipo de la union */
union alfa { int x;
            char y[10];
            float z;};
/* Se declaran los prototipos de las funciones */
int suma(int ,int );
int ingreso(void);
void main(void)
{
    /* Se declaran las variables de tipo union */
    union alfa A,B;
    clrscr();
    /* Se invoca a la función "ingreso" */
    A.x=ingreso();
    B.x=ingreso();
    printf("\n%d+%d=",A.x,B.x);
    /* Se invoca a la función "suma" */
    A.x=suma(A.x,B.x);
    printf("%d\n",A.x);
    /* Se limpia el buffer del teclado */
    fflush(stdin);
    gets(A.y);
    if(A.y[2]>'F')
    {
        A.z=3.14;
        scanf("%f",&B.z);
        printf("\n\nPerimetro = %f",A.z*B.z);
    }
}
/* Se define a la función "ingreso"      *
 * ésta devuelve un valor de tipo "int" *
int ingreso(void)
{
    int a;
    scanf("%d",&a);
    return(a);
}
/* Se define a la función "suma"      *
 * ésta devuelve un valor de tipo "int" *
 * y recibe dos parámetros de tipo "int" *
int suma(int a,int b)
{
    int c;
    c=a+b;
    return(c);
}
```

En este ejemplo se define una unión que contiene tres campos, uno de tipo "int" denominado "x", otro de tipo "char" llamado "y" que es un vector de 10 posiciones y un campo "float", cuyo nombre es "z".

Primero se asignan al campo "x" de las variables "A" y "B" los valores que retorna la función "ingreso", esta permite la entrada de un valor "int" utilizando la función "scanf".

Luego se envían a la función "suma" el campo "x" de las uniones "A" y "B" y el resultado de ésta se le asigna al campo "x" de la variable "A", el valor devuelto por esta función es el resultado de sumar ambos parámetros.

PASAJE Y RETORNO DE UNIONES A FUNCIONES

Se puede realizar la transferencia de una unión como parámetro de una función procediendo de forma similar a lo que se realiza con una estructura, indicando el tipo de la unión en la declaración y prototipo de la función y, además, la declaración de la misma debe realizarse en forma global, dado que si así no se realizase no sería vista por las funciones a las cuales se les transfiere su valor.

De igual forma se puede retornar una unión desde una función, indicando el tipo de la misma como parámetro de retorno de ella. En la sentencia "return" dentro de la función se debe retornar una variable del tipo de la unión utilizada para retornar.

Hay que tener cuidado de que solo se pueden pasar y retornar uniones del mismo tipo hacia y desde una función.

Ejemplo

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
/* Se define una unión con dos campos uno "int" y el otro "char" */
union alfa { int x;
             char y;};
/* Se declaran los prototipos de las funciones */
union alfa ingreso(char);
void imprime(union alfa,char);
void main(void)
{
    /* Se declaran las variables */
    union alfa A;
    char c;
    do
    {
        /* Menú de opciones */
        clrscr();
        printf("\n\n\t\t\tIngrese una opción ");
        printf("\n\n (C) ingreso de un carácter");
        printf("\n\n (E) ingreso de un entero ");
        c=getche();
        printf("\n\n");
        /* Se invoca a la función "ingreso" */
        A=ingreso(c);
        /* Se invoca a la función "imprime" */
        imprime(A,c);
        printf("\n\n\nDesea ingresar otro valor (S/N) ");
        c=getche();
    }
    while((c=='s')||(c=='S'));
}
```



```

/* Se define a la función "ingreso" *
 * devuelve una unión de tipo "alfa" *
 * recibe un "char" */
union alfa ingreso(char b)
{
    union alfa C;
    printf("Ingrese un ");
    if((b=='c')||(b=='C'))
    {
        printf(" carácter ");
        C.y=getche();
    }
    else
    {
        printf(" entero ");
        scanf("%d",&C.x);
    }
    printf("\n\n");
    /* Devuelve el valor de la unión */
    return(C);
}

/* Se define la función "imprime" *
 * recibe una unión de tipo alfa llamada "B" *
 * y un char denominado "c", devuelve "void" */
void imprime(union alfa B,char c)
{
    if((c=='c')||(c=='C'))
        printf("\n\nUsted ingreso el carácter %c",B.y);
    else
        printf("\n\nUsted ingreso el entero %d",B.x);
}

```

En este ejemplo se puede ver como se recibe una unión como valor retornado por una función y como una función recibe como parámetro el valor de una unión.

En la función "ingreso" se observa como es el procedimiento de retorno de una unión, esta es invocada en "main" y el valor retornado se le asigna a una unión denominada "A".

En "main" también se llama a la función "imprime" pasándole los parámetros en los cuales uno es del tipo de la unión "alfa" mediante el envío de la variable "A".

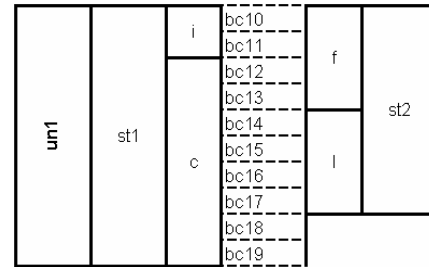
UNIONES DE ESTRUCTURAS

Una unión puede contener como campos cualquier tipo de dato válido, por lo que dentro de ella también pueden existir una o varias estructuras. Cuando existen estructuras dentro de una unión es lo que se llama "unión de estructuras".

En este caso los campos de la unión se encuentran empaquetados dentro de cada estructura, ahora el tamaño de la unión está dado por la estructura mayor, aquella que ocupa mayor cantidad de bytes en la memoria.

Ahora las estructuras comienzan en la misma dirección, solapándose una a la otra. Veamos un ejemplo de este tipo de uniones.

```
union un1 { struct { int i;
                  char c[8];} st1;
            struct { float f;
                  long l;} st2;} un1
```



Podemos observar que no se solapan las variables de cada estructura entre sí, pero si lo hacen ambas estructuras.

Debemos recordar que para acceder a un campo de la unión se debe utilizar el operador punto. Utilizando uno solo accederíamos a una estructura. Para acceder a una variable deberíamos utilizar dos operadores punto, el primero para acceder a la estructura y el segundo para poder referenciar el campo de la estructura.

Si deseamos referirnos al campo "l" de la estructura "st2" que pertenece a la unión "un1" lo debemos hacer de la siguiente forma:

```
un1.st2.l=948;
```

Ejemplo

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
/* Largo máximo del vector y largo de la pantalla */
#define MAX 100
#define PANT 6
/* Definición de las estructuras que integran a la unión */
struct tiem { unsigned char min,seg;};
struct libros { char ident;
                unsigned int num;
                char titu[50],autor[50],edit[20];
                unsigned int pag;};
struct discos { char ident;
                unsigned int num;
                char inte[30],gene[20];
                unsigned char tem;
                struct tiem dur;};
/* Definición de la unión de estructuras */
union datos { struct libros lib;
              struct discos dis;};
/* Declaración de los prototipos de las funciones */
void bolib(union datos indice[]);
void inlib(union datos indice[]);
void lilib(union datos indice[]);
void indis(union datos indice[]);
void bodis(union datos indice[]);
void lidis(union datos indice[]);
/* Declaración de una variable global e inicialización de la misma */
int cant=0;
```



```

        if((c=='s')||(c=='S'))
        {
            for(j=i;j<cant;j++)
                indice[j]=indice[j+1];
            cant--;
        }
    }
    printf("\n\nDesea borrar otro libro (S/N)? ");
    c=getche();
}
while((c=='s')||(c=='S'));
}
/* Definición de la función "inlib" que se encargara de ingresar *
 * uno o mas libros */
void inlib(union datos indice[])
{
    struct libros L;
    int i;
    char c;
    do
    {
        if(cant<MAX)
        {
            L.ident='L';
            for(i=cant;(indice[i].lib.ident!='L')&&i--);
            if(i)
                L.num=indice[i].lib.num+1;
            else
                L.num=1;
            do
            {
                clrscr();
                fflush(stdin);
                printf("\n\tIngrese el titulo del libro ");
                gets(L.titu);
                printf("\n\tIngrese el autor del libro ");
                gets(L.autor);
                printf("\n\tIngrese la editorial del libro ");
                gets(L.edit);
                printf("\n\tIngrese la cantidad de páginas ");
                printf("del libro ");
                scanf("%u",&L.pag);
                printf("\n\nLos datos son correctos (S/N) ? ");
                c=getche();
            }
            while((c!='s')&&(c!='S'));
            cant++;
            indice[cant].lib=L;
        }
        else
            printf("\n\nNo puede ingresar un nuevo libro");
        printf("\n\nDesea ingresar otro libro (S/N)? ");
        c=getche();
    }
    while((c=='s')||(c=='S')&&(cant<MAX));
}

```

```

/* Definición de la función "lilib" que se encargara de listar *
 * los libros */
void lilib(union datos indice[])
{
    int i,j;
    for(i=0;i<cant;i+=PANT)
    {
        clrscr();
        for(j=0;(j<PANT)&&((i+j)<=cant);j++)
        {
            if(indice[i+j].lib.ident=='L')
            {
                printf("\n\nNúmero : %d ",indice[i+j].lib.num);
                printf("Titulo : %s ",indice[i+j].lib.titu);
                printf("\nAutor : %s ",indice[i+j].lib.autor);
                printf("Editorial : %s ",indice[i+j].lib.edit);
                printf("Cantidad de páginas : %u ",\
                    indice[i+j].lib.pag);
            }
        }
        printf("\n\nPresione una tecla para continuar");
        getch();
    }
}

/* Definición de la función "indis" que será la encargada de *
 * ingresar uno o mas discos */
void indis(union datos indice[])
{
    struct discos L;
    int i;
    char c;
    do
    {
        if(cant<MAX)
        {
            L.ident='D';
            for(i=cant;(indice[i].dis.ident!='D')&&i;i--);
            if(i)
                L.num=indice[i].dis.num+1;
            else
                L.num=1;
            do
            {
                clrscr();
                fflush(stdin);
                printf("\n\tIngresa el interprete del disco ");
                gets(L.inte);
                printf("\n\tIngresa el genero del disco ");
                gets(L.gene);
                printf("\n\tIngresa la cantidad de temas ");
                printf("del disco ");
                scanf("%u",&L.tem);
                printf("\n\tIngresa la duración del ");
                printf("disco (mm:ss) ");
                scanf("%u:%u",&L.dur.min,&L.dur.seg);
                printf("\n\nLos datos son correctos (S/N) ? ");
                c=getche();
            }
        }
    }
}

```

```

        while((c!='s')&&(c!='S'));
        cant++;
        indice[cant].dis=L;
    }
    else
        printf("\n\nNo puede ingresar un nuevo disco");
    printf("\n\nDesea ingresar otro disco (S/N)? ");
    c=getche();
}
while((c=='s')||(c=='S')&&(cant<MAX));
}
/* Definición de la función "bodis" que será la encargada de borrar *
 * uno o mas discos */
void bodis(union datos indice[])
{
    char c,titu[50];
    int i,j;
    do
    {
        clrscr();
        printf("\t\tIngresa el interprete del disco ");
        fflush(stdin);
        gets(titu);
        for(i=0;(i<=cant)&&(strcmp(indice[i].lib.titu,titu));i++);
        if(i>cant)
            printf("\n\n\t\tEl disco no fue encontrado ");
        else
        {
            printf("\n\n\t\tGenero : %s",indice[i].dis.gene);
            printf("\n\nEsta seguro (S/N)? ");
            c=getche();
            if((c=='s')||(c=='S'))
            {
                for(j=i;j<cant;j++)
                    indice[j]=indice[j+1];
                cant--;
            }
        }
        printf("\n\nDesea borrar otro disco (S/N)? ");
        c=getche();
    }
    while((c=='s')||(c=='S'));
}
/* Definición de la función "lidis" que será la *
 * encargada de listar en pantalla a los discos */
void lidis(union datos indice[])
{
    int i,j;
    for(i=0;i<cant;i+=PANT)
    {
        clrscr();
        for(j=0; (j<PANT)&&((i+j)<=cant);j++)
        {
            if(indice[i+j].dis.ident=='D')
            {
                printf("\n\nNúmero : %d ",indice[i+j].dis.num);
                printf("Interprete : %s ",indice[i+j].dis.inte);
                printf("\nGenero : %s ",indice[i+j].dis.gene);
                printf("Temas : %u ",indice[i+j].dis.tem);
            }
        }
    }
}

```

```

        printf("Duración : %u:%u ",\
               indice[i+j].dis.dur.min,\
               indice[i+j].dis.dur.seg);
    }
}
printf("\n\nPresione una tecla para continuar");
getch();
}

```

Se puede observar la utilización de una unión de estructuras para guardar los datos de dos objetos diferentes, como ser libros y discos. Estos tienen características particulares que no permiten poder guardarlos en un único vector.

Se utilizan 6 funciones para poder realizar las diferentes operaciones elegidas en el menú de opciones que se despliega en la función “main”. En éstas se permite el ingreso de datos, eliminar datos y el listado de los mismos.

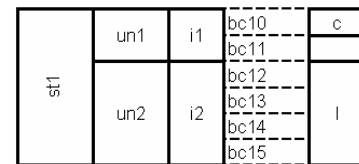
ESTRUCTURAS DE UNIONES

En este caso debemos hacer referencia a una estructura en la cual algunos o todos los pueden ser uniones. Utilizando este tipo de datos se puede ver que los campos que pertenecen a la estructura son uniones y se ubican una a continuación de la otra. Las variables que se solapan son las que están dentro de cada unión.

```

struct stun { union {int i1;
                    char c;}un1;
              union {int i2[2];
                    long l;}un2;} st1;

```



Encontramos en el ejemplo anterior que las variables "c" e "i1" y "l" e "i2" están ocupando las mismas posiciones de memoria, pero la unión "un1" está ubicada a continuación de "un2".

Para referenciar un campo de la estructura se debe utilizar el operador punto, al igual que en la unión de estructuras se usará un operador para invocar a una unión y dos operadores punto para invocar a la variable de la unión.

```
st1.un2.i2[1]=65;
```

En el ejemplo encontramos que se asigna el valor 65 al campo "i2[1]" de la unión "un2" de la estructura "st1".

Ejemplo

```
#include<stdio.h>
#include<conio.h>
/* MAX determina el largo máximo del vector */
#define MAX 100
/* Se definen las estructuras y uniones a utilizar */
struct coupe { char nro_cil;
               int cilind,veloc_max;
               char marchas;};
struct sedan { char puertas,color[10],familiar;};
struct cuotas { int cant;
               float val;};
union clase { struct coupe dat;
              struct sedan dat1;};
union fina { float contado;
            struct cuotas pagos;};
struct autos { char marca[10];
              int modelo;
              char tipo;
              union clase caract;
              char vendido,cont;
              union fina valor;};
/* Se declaran los prototipos de las funciones */
int ingreso(struct autos stock[],int);
void listado(struct autos stock[],int);
void main(void)
{
    /* Se declaran las variables a utilizar */
    struct autos stock[MAX];
    int i,j=0;
    /* Impresión del menú de opciones */
    do
    {
        clrscr();
        printf("\n\t\t\tMenú principal");
        printf("\n\t1 - Ingreso de datos");
        printf("\n\t2 - Listado de datos");
        printf("\n\t3 - Salir");
        printf("\n\nIngrese una opción ( 1 - 3 ) ");
        scanf("%d",&i);
        switch(i)
        {
            case 1 : j=ingreso(stock,j);
                    break;
            case 2 : listado(stock,j);
        }
    }
    while(i!=3);
}
```



```

/* Función para permitir el ingreso de los datos */
int ingreso(struct autos stock[],int j)
{
    /* Declaración de las variables a utilizar para *
    * realizar el ingreso de los valores de tipo *
    * "char" con formato numérico y para poder *
    * ingresar los valores de tipo "float" */
    int i;
    float f;
    /* Verifica que no se exceda el máximo del vector */
    if(j==MAX)
        return(j);
    /* Comienzo del ingreso de los datos */
    do
    {
        clrscr();
        printf("\n\t\t\t\t\tINGRESO DE DATOS");
        printf("\n\n\t\t\t\t\tMarca : ");
        fflush(stdin);
        gets(stock[j].marca);
        printf("\t\t\t\t\tModelo : ");
        scanf("%d",&stock[j].modelo);
        fflush(stdin);
        printf("\t\t\t\t\tTipo (C-S) : ");
        /* Ingreso de datos selectivo según el tipo elegido */
        do
        {
            stock[j].tipo=getche();
            if(stock[j].tipo=='C' || stock[j].tipo=='c')
            {
                printf("\n\t\t\t\t\tNúmero de cilindros : ");
                /* Utilización de una variable auxiliar para *
                * poder ingresar un valor con formato *
                * numérico en una variable de tipo "char" */
                scanf("%d",&i);
                stock[j].caract.dat.nro_cil=i;
                printf("\t\t\t\t\tCilindrada : ");
                scanf("%d",&stock[j].caract.dat.cilind);
                printf("\t\t\t\t\tVelocidad máxima : ");
                scanf("%d",&stock[j].caract.dat.veloc_max);
                printf("\t\t\t\t\tCantidad de marchas : ");
                scanf("%d",&i);
                stock[j].caract.dat.marchas=i;
            }
            if(stock[j].tipo=='S' || stock[j].tipo=='s')
            {
                printf("\n\t\t\t\t\tCantidad de puertas : ");
                scanf("%d",&i);
                stock[j].caract.dat1.puertas=i;
                fflush(stdin);
                printf("\t\t\t\t\tColor : ");
                gets(stock[j].caract.dat1.color);
                printf("\t\t\t\t\tFamiliar (S/N) : ");
                stock[j].caract.dat1.familiar=getche();
                printf("\n");
            }
        }
    }
}

```

```

while(!(stock[j].tipo!='C' || stock[j].tipo!='c' || \
stock[j].tipo!='S' || stock[j].tipo!='s'));
printf("\tVendido (S/N) : ");
/* Ingreso selectivo de los datos de venta */
do
{
    stock[j].vendido=getche();
    if(stock[j].vendido=='S' || stock[j].vendido=='s')
    {
        printf("\n\tVenta al contado (S/N) : ");
        do
        {
            stock[j].cont=getche();
            if(stock[j].cont=='S' || stock[j].cont=='s')
            {
                printf("\n\tValor : ");
                /* Utilización de una variable *
                * auxiliar para permitir el *
                * ingreso de una variable de *
                * tipo "float" en un vector de *
                * estructuras */
                scanf("%f",&f);
                stock[j].valor.contado=f;
            }
            if(stock[j].cont=='N' || stock[j].cont=='n')
            {
                printf("\n\tNúmero de cuotas : ");
                scanf("%d",&stock[j].valor.pagos.cant);
                printf("\tValor de la cuota : ");
                scanf("%f",&f);
                stock[j].valor.pagos.val=f;
            }
        }
        while(!(stock[j].cont!='S' || stock[j].cont!='s' \
        || stock[j].cont!='N' || stock[j].cont!='n'));
    }
    /* En caso de no haberse producido la venta se *
    * inicializan las variables a cero */
    if(stock[j].vendido=='N' || stock[j].vendido=='n')
    {
        stock[j].valor.pagos.cant=0;
        stock[j].valor.pagos.val=0;
    }
}
while(!(stock[j].vendido!='S' || stock[j].vendido!='s' || \
stock[j].vendido!='N' || stock[j].vendido!='n'));
printf("\n\t\tDesea ingresar otro vehículo (S/N) ");
i=getche();
j++;
}
/* Se comprueba que no se exceda el largo máximo del vector en *
* el caso de que se quieran agregar mas datos que los *
* permitidos */
while(i=='S' || i=='s' && j<MAX);
/* Se retorna el último valor de la posición máxima de los *
* valores ingresados */
return(j);
}

```

```

/* Función que permite la visualización de los datos */
void listado(struct autos stock[],int i)
{
    /* Declaración de las variables a utilizar en la función */
    int j,k;
    /* Lazo para poder realizar la visualización de los datos *
    * dentro del rango ingresado */
    for(j=0;j<i;j++)
    {
        clrscr();
        printf("\n\n\tMarca : %s",stock[j].marca);
        printf("\tModelo : %d",stock[j].modelo);
        printf("\tTipo : %c\n",stock[j].tipo);
        /* Listado selectivo según el tipo */
        if(stock[j].tipo=='C' || stock[j].tipo=='c')
        {
            printf("\n\tNúmero de cilindros : %d"\
                ,stock[j].caract.dat.nro_cil);
            printf("\n\tCilindrada : %d",\
                stock[j].caract.dat.cilind);
            printf("\n\tVelocidad máxima : %d"\
                ,stock[j].caract.dat.veloc_max);
            printf("\n\tCantidad de marchas : %d",\
                stock[j].caract.dat.marchas);
        }
        if(stock[j].tipo=='S' || stock[j].tipo=='s')
        {
            printf("\n\tCantidad de puertas : %d"\
                ,stock[j].caract.dat1.puertas);
            printf("\n\tColor : %s",stock[j].caract.dat1.color);
            printf("\n\tFamiliar : %c"/
                ,stock[j].caract.dat1.familiar);
        }
        printf("\n\n\tVendido : %c",stock[j].vendido);
        /* Listado selectivo de los datos de venta */
        if(stock[j].vendido=='S' || stock[j].vendido=='s')
        {
            if(stock[j].cont=='S' || stock[j].cont=='s')
                printf("\n\tValor : %.2f",\
                    stock[j].valor.contado);
            if(stock[j].cont=='N' || stock[j].cont=='n')
            {
                printf("\n\tNúmero de cuotas : %d"\
                    ,stock[j].valor.pagos.cant);
                printf("\tValor de la cuota : %.2f"\
                    ,stock[j].valor.pagos.val);
            }
        }
        printf("\n\n\t\tDesea listar otro vehículo (S/N) ");
        k=getche();
        /* Selección que permite seguir visualizando o no los *
        * datos del vector según la elección del operador */
        if(k=='N' || k=='n')
            return;
    }
}

```

Este ejemplo permite realizar el ingreso y la visualización de los datos de vehículos para poder realizar las ventas de los mismos. Utiliza dos funciones para ello una llamada “ingreso” y la otra denominada “listado” ambas reciben el vector y la cantidad de datos ingresados hasta el momento.

El vector en donde se ingresan los datos es un vector de estructuras en el cual dos de los campos utilizados son uniones, éstas se utilizan para guardar los datos según la característica elegida, tipo de automóvil o forma de la venta.

Las uniones se denominan “clase” y “fina”. La unión “clase” se compone de dos campos “dat” y “dat1” que a su vez son estructuras, las que se le llaman “coupe” y “sedan”, ellas guardaran los datos correspondientes a las características particulares de cada tipo.

La unión “fina” contiene dos campos, uno de tipo “float” y el otro una estructura que permite guardar los datos correspondientes a la cantidad de cuotas y el valor de cada cuota.

Se utilizan variables auxiliares para permitir el ingreso de valores en formato numérico a variables de tipo “char” y para poder realizar el ingreso de datos de tipo “float”

CAMPOS DE BITS

Un campo de bits es una estructura particular que permite asignar valores binarios a cada campo de la estructura.

Cada campo de la estructura es una variable binaria del largo que se fije según se desee.

DECLARACIÓN

La declaración de un campo de bits se realiza de una de tres diferentes formas.

```
struct etiqueta {   tipo entero campo_1: cantidad de bits;  
                  tipo entero campo_2: cantidad de bits;  
                  tipo entero campo_3: cantidad de bits;  
                  ...  
                  ...  
                  tipo entero campo_n: cantidad de bits;} lista_de_variables;
```

```
struct { tipo entero campo_1: cantidad de bits;  
          tipo entero campo_2: cantidad de bits;  
          tipo entero campo_3: cantidad de bits;  
          ...  
          ...  
          tipo entero campo_n: cantidad de bits;} lista_de_variables;
```

```
struct etiqueta {   tipo entero campo_1: cantidad de bits;  
                  tipo entero campo_2: cantidad de bits;  
                  tipo entero campo_3: cantidad de bits;  
                  ...  
                  ...  
                  tipo entero campo_n: cantidad de bits;};
```

Podemos observar que la diferencia entre la declaración de una estructura y un campo de bits consiste en que solamente se puede utilizar un tipo de dato entero para declarar al campo y que se especifica el largo en bits de dicho campo.

Hay que tener en cuenta que la máxima cantidad de bits que se pueden asignar solo depende del tamaño de la variable entera que se utiliza en su definición.

En todo lo demás se comporta como una estructura.

En referencia al tipo, se pueden utilizar el "char", "int" o "long" pero no son convenientes debido a que el valor que se observará cuando el bit más significativo tome el valor uno será un valor distinto al deseado, esto es debido a que cuando ocurre tal caso en una variable entera, este bit corresponde al signo de dicha variable. Para solucionar este aspecto es conveniente utilizar el modificador "unsigned".

El espacio asignado a cada uno de los campos de bits depende del tamaño del tipo asociado al campo de bits, de tal forma que ubica cada uno de los campos en forma sucesiva hasta llegar a ocupar todos los bits del largo del tipo asociado comenzado desde el bit menos significativo al más significativo. Hay que tener en cuenta que este proceso no es portable, entonces deberemos determinar la forma en que se ubican dichos campos en cada sistema en particular, en el caso este proceso no fuese de esta forma lo será en forma inversa pudiendo corregirlo durante el proceso.

Supongamos el siguiente campo de bits:

```
Struct datos { unsigned char A:3;
               unsigned char B:4;
               unsigned char C:1;};
```

El tamaño total de la estructura sería de un byte, debido a que el total de los bits individuales de cada uno de los campos es igual al largo de un char (1 byte). Estos se ubicarían de estas posibles formas :



Si la cantidad de bits definidos es menor al del tipo asociado, estos quedarán vacíos. Si ocurriese que una variable no cupiese en el lugar disponible se agregaría un nuevo tamaño del tipo asociado.

Ejemplo

```
#include <stdio.h>
struct dato { unsigned int A:5;
              unsigned int B:9;} valores;
float valmed(void);
void main(void)
{
    struct dato ingreso;
    -----
    -----
    -----
}
```

```
float valmed(void);
{
    struct dato aux;
    -----
    -----
    -----
}
```

Aquí podemos encontrar que la estructura dato se define como un campo de bits, en ésta se encuentra que se define un campo de un ancho de 5 bits denominado "A" y otro de un ancho de 9 bits llamado "B".

También encontramos que se define una variable global de este tipo, "valores" y dos variables locales una a "main" y la otra a "valmed".

```
#include <stdio.h>
struct { unsigned char A:3;
        unsigned char B:1;
        unsigned char C:2;}  calc;
void soluc(void);
void main(void)
{
    struct { unsigned char A:3;
            unsigned char B:1;
            unsigned char C:2;}  datos[25];
    -----
    -----
    -----
}
void soluc(void)
{
    struct { unsigned char A:3;
            unsigned char B:1;
            unsigned char C:2;}clave;
    -----
    -----
    -----
}
```

En este se definen tres campos de bits idénticos entre sí, pero como no se le asigno una etiqueta a cada uno de ellos, los campos de bits no son del mismo tipo.

El campo "A" tiene una longitud de 3 bits, el "B" tan solo de un bit y el "C" de dos bits.

```
#include <stdio.h>
struct cpo1 { unsigned long A:13;
             unsigned long B:15;
             unsigned long C:7;};
void disposit(void);
void main(void)
{
    struct cpo1 clase;
    -----
    -----
    -----
}
```

```

void disposit(void)
{
    struct cpo1 sensado;
    -----
    -----
    -----
}

```

Podemos ver aquí que se define un tipo de dato de forma global que contiene tres campos de bits, uno de 13, otro de 15 y el ultimo de 7.

Todas las variables fueron declaradas locales a las funciones del tipo "struct cpo1".

REFERENCIA Y ASIGNACIÓN DE VALORES EN UN CAMPO DE BITS

Para poder referenciar un valor en un campo de bits es necesario, primero escribir el nombre de la estructura que contiene el campo de bits a utilizar, luego escribir el operador punto y por último invocar el nombre del campo de bits que se quiera utilizar. Este procedimiento es el mismo que se utiliza para llamar a un campo en una estructura. Cuando se quiere asignar un valor a un campo de bits se debe utilizar el operador de agnación (=) escribiendo en el lado derecho del operador el valor numérico deseado o la variable que se le quiere asignar.

Hay que tener la precaución de utilizar una variable para asignársela a un campo de bits, únicamente se puede realizar esta operación cuando se asignan campos de igual tamaño.

Ejemplo

```

#include<stdio.h>
#include<conio.h>

/* Se definen los campos de bits */

struct alfa    { unsigned char A:1;
                unsigned char B:2;
                unsigned char C:3;};

struct beta    { unsigned char A:3;
                unsigned char B:2;
                unsigned char C:1;};

struct gamma   { unsigned char A:5;
                unsigned char B:2;
                unsigned char C:3;
                unsigned char D:4;};

union delta    { struct alfa D;
                struct beta E;
                struct gamma F;
                char G[2];};

```

```

void main(void)
{
    char a;
    /* Se declara un campo de bits y una unión para visualizar el *
     * comportamiento de los campos de bits */
    struct alfa b;
    union delta e;
    clrscr();
    /* Se examinan los valores posibles de las variables binarias *
     * verificando que en caso de excederse el máximo valor posible *
     * vuelve al primer valor válido */
    for(b.A=0,a=0;a<4;b.A++,a++)
        printf("%d  ",b.A);
    printf("\n");
    for(b.B=0,a=0;a<8;b.B++,a++)
        printf("%d  ",b.B);
    printf("\n");
    for(b.C=0,a=0;a<16;b.C++,a++)
        printf("%d  ",b.C);
    printf("\n\nPresione una tecla para continuar ");
    getch();
    clrscr();
    /* Limpia el contenido del vector de la unión y lo imprime */
    e.G[0]=0;
    e.G[1]=0;
    printf("\ne.g[0]=%d  e.G[1]=%d",e.G[0],e.G[1]);
    /* Asigna valores a los campos de bits y los imprime */
    e.D.A=1;
    e.D.B=2;
    e.D.C=5;
    printf("\ne.D.A=%d  e.D.A=%d  e.D.C=%d",e.D.A,e.D.B,e.D.C);
    printf("\ne.G[0]=%X  e.G[1]=%X",e.G[0],e.G[1]);
    e.E.C=1;
    e.E.B=2;
    e.E.A=5;
    printf("\ne.E.A=%d  e.E.A=%d  e.E.C=%d",e.E.A,e.E.B,e.E.C);
    printf("\ne.G[0]=%X  e.G[1]=%X",e.G[0],e.G[1]);
    e.F.A=18;
    e.F.B=2;
    e.F.C=5;
    e.F.D=11;
    printf("\ne.F.A=%X  e.F.A=%X  ",e.F.A,e.F.B);
    printf("e.F.C=%X  e.F.D=%X",e.F.C,e.F.D);
    printf("\ne.G[0]=%X  e.G[1]=%X",e.G[0],e.G[1]);
    printf("\n\nPresione una tecla para continuar ");
    getch();
}

```

Vemos en este ejemplo como se puede asignar un valor y referenciarse a un campo de bits. También se puede observar que las variables del tipo de campo de bits pueden tomar solo los valores permitidos según su tamaño, cuando la variable supera su valor máximo automáticamente vuelve a comenzar con el valor mínimo.

Se puede verificar como se ubican los campos de bits en la memoria mediante el uso de una unión, revisando los valores que se le ingresan a las variables binarias e imprimiendo los bytes correspondientes.

OPERADORES A NIVEL DE BITS

Los operadores a nivel de bits son aquellos que trabajan modificando los valores de los bits individuales, es decir, que actúan bit a bit sobre el valor binario guardado en la posición de memoria sobre la que trabajan.

Los operadores convencionales se diferencian de éstos porque actúan sobre la variable completa, evaluando su contenido sin importar los estados de los bits que la componen.

Operador and (&)

El operador and (&) realiza la función del producto lógico entre dos elementos pero analizado bit a bit. Se diferencia del operador relacional and (&&) que también realiza la misma operación, pero este último lo ejecuta evaluando el contenido completo de la variable y no lo realiza analizando cada par de bits en particular.

Recordemos que el resultado de una función "and" es el siguiente.

<i>a</i>	<i>b</i>	<i>a & b</i>
0	0	0
0	1	0
1	0	0
1	1	1

Este operador tiene la ventaja de poder poner al descubierto el o los bits deseados en una variable utilizando la máscara adecuada (valor numérico elegido por el usuario para poder realizar la tarea deseada) y poniendo en cero los bits que carecen de interés.

Operador or (|)

Este realiza la función lógica or (suma lógica), evaluando los operandos bit a bit. Se diferencia del operador relacional or (||) dado que éste analiza a los operandos según su contenido total.

Reveamos los valores que toma la función "or"

<i>a</i>	<i>b</i>	<i>a b</i>
0	0	0
0	1	1
1	0	1
1	1	1

Posee también la capacidad de poder enmascarar bits en forma individual, poniendo en uno los bits no deseados y dejando con el valor que tenían aquellos que son de interés.

Operador xor (^)

Este operador realiza la función lógica or exclusiva.
También permite enmascarar bits al igual que los anteriores.
Su resultado es el siguiente:

a	b	$a \wedge b$
0	0	1
0	1	0
1	0	0
1	1	1

Operador not (~)

Realiza la negación de los bits correspondientes, éstos invierten su valor. Cuando un bit tiene el valor uno al negarlo se transforma en cero.
El efecto de esta función es el siguiente:

a	$\sim a$
0	1
1	0

Operador desplazamiento izquierdo (<<)

Este operador realiza el desplazamiento de los bits de una variable a la izquierda perdiendo el contenido del bit más significativo y agregando ceros a los bits ubicados a la derecha. La cantidad de bits a desplazar depende del valor indicado a la derecha del símbolo <<. Esta operación debe utilizarse siempre a la izquierda del operador de asignación (=).

Operador desplazamiento derecho (>>)

Realiza el desplazamiento de la cantidad de bits a la derecha, perdiendo el bit menos significativo e insertando cero a los bits de la izquierda.
En esta operación se debe utilizar siempre con el símbolo de asignación (=).

Ejemplo

```
#include<stdio.h>
#include<conio.h>
void main (void)
{
    unsigned int nro,aux;
    clrscr();
    do
    {
        clrscr();
        printf("\n\n\t\tIngrese un número entero : ");
        scanf("%d",&nro);
        printf("\n\n\t\tEl valor en binario es : ");
        /* Imprime un número en formato binario *
         * para el cálculo utiliza una máscara y *
         * utiliza para ello la operación and *
         * y el desplazamiento de bits */
        for(aux=0x8000;aux>0;aux>>=1)
            if(nro&aux)
                printf("1");
            else
                printf("0");
        printf("\n\nDesea continuar (S/N)? ");
    }
    while(getche()!='N');
}
```

En este ejemplo se obtiene la representación de un número en formato binario. Para ello se realiza una operación "and" entre el número dado y una máscara que tiene solo un bit en uno y el resto con valor cero.

Se procede a comprobar el valor del resultado de dicha operación e imprimir un uno en caso de que la comparación sea verdadera y un cero en caso contrario. Se va desplazando el uno de la máscara hacia la derecha en un lugar de forma tal de poder escribir el número binario en el orden correcto, desde el bit más significativo hasta el menos significativo.

```
#include<stdio.h>
#include<conio.h>
void main (void)
{
    int i,j;
    clrscr();
    do
    {
        clrscr();
        printf("\n\n\t\tIngrese un número entero : ");
        scanf("%d",&i);
        /* Imprime el número ingresado en formato hexadecimal */
        printf("\n\t\tEl número en hexadecimal es %04X",i);
        /* Calcula el complemento a dos del número ingresado */
        j=~i+1;
    }
```

```

        /* Imprime el número en complemento a dos en el formato *
        * decimal y en el formato hexadecimal */
        printf("\n\n\t\tEl número en complemento a dos es %d",j);
        printf("\n\n\t\tEn formato hexadecimal es %04X",j);
        printf("\n\n\nDesea continuar (S/N)? ");
    }
    while(getche()!='N');
}

```

En este se muestra como convertir un número en su correspondiente valor en complemento a dos. Para ello se utiliza el operador "not" y luego se le suma uno al resultado de la operación de negación del número correspondiente.

Esta notación se utiliza en lenguaje "C" para expresar un número negativo. La representación del número ingresado y de su resultado en complemento a dos se expresan en formato decimal y en formato hexadecimal. Se puede observar que los resultados en formato decimal son iguales en modulo pero de signo contrario; en cambio en formato hexadecimal se encuentra que los resultados son distintos entre sí pero hay que ver que ambos difieren en el bit más significativo.

```

#include<stdio.h>
#include<conio.h>
void conv(unsigned char,char,unsigned char,unsigned char);
void conv1(char,unsigned char,unsigned char);
void conv2(unsigned char,char,unsigned char,unsigned char);
void pasbin(unsigned char);
void main(void)
{
    unsigned char a,i,b,c;
    do
    {
        clrscr();
        printf("\n\n\t\tIngrese un número entero : ");
        scanf("%d",&a);
        printf("\n\n\t\tIngrese otro número entero : ");
        scanf("%d",&b);
        /* Calcula todas las operaciones a nivel de bit entre *
        * los dos números ingresados */
        printf("\n\n\t\t%d | %d = %d",a,b,a|b);
        conv(a,'|',b,a|b);
        printf("\n\n\t\t%d & %d = %d",a,b,a&b);
        conv(a,'&',b,a&b);
        printf("\n\n\t\t%d ^ %d = %d",a,b,a^b);
        conv(a,'^',b,a^b);
        printf("\n\n\t\t~%d = %d",a,~a);
        conv1('~',a,~a);
        printf("\n\n\t\t~%d = %d",b,~b);
        conv1('~',b,~b);
        printf("\n\n\t\t\tPresione una tecla para continuar");
        getch();
        clrscr();
        printf("\n\n");
    }
}

```

```

/* Calcula los desplazamientos a derecha *
 * de los valores ingresados */
for(i=0;i<9;i++)
{
    c=a>>i;
    printf("\n\t\t%d >> %d = %d ",a,i,c);
    conv2(a,'>',i,c);
}
printf("\n\n");
for(i=0;i<9;i++)
{
    c=b>>i;
    printf("\n\t\t%d >> %d = %d ",b,i,c);
    conv2(b,'>',i,c);
}
printf("\n\n\t\t\tPresione una tecla para continuar");
getch();
clrscr();
printf("\n\n");
/* Calcula los desplazamientos a izquierda *
 * de los valores ingresados */
for(i=0;i<9;i++)
{
    c=a<<i;
    printf("\n\t\t%d << %d = %d ",a,i,c);
    conv2(a,'<',i,c);
}
printf("\n\n");
for(i=0;i<9;i++)
{
    c=b<<i;
    printf("\n\t\t%d << %d = %d ",b,i,c);
    conv2(b,'<',i,c);
}
printf("\n\nDesea continuar (S/N)? ");
}
while(getche()!='N');
}
/* Escribe un número en formato binario */
void pasbin(unsigned char nro)
{
    unsigned char aux;
    for(aux=0x80;aux>0;aux>>=1)
        if(nro&aux)
            printf("1");
        else
            printf("0");
}
/* Rutina de impresión de las operaciones en formato binario */
void conv(unsigned char a,char b,unsigned char c,unsigned char d)
{
    printf("\t\t");
    pasbin(a);
    printf(" %c ",b);
    pasbin(c);
    printf(" = ");
    pasbin(d);
}

```

```

/* Rutina de impresión de las operaciones en formato binario */
void conv1(char a,unsigned char b,unsigned char c)
{
    printf("\t\t%c",a);
    pasbin(b);
    printf(" = ");
    pasbin(c);
}
/* Rutina de impresión de las operaciones en formato binario */
void conv2(unsigned char a,char b,unsigned char c,unsigned char d)
{
    printf("\t\t");
    pasbin(a);
    printf(" %c%c %d = ",b,b,c);
    pasbin(d);
}

```

En este podemos ver los resultados de aplicar las operaciones a nivel de bit entre dos números que se ingresan desde el teclado. Estos valores se expresan tanto en formato decimal como en forma binaria. Para expresar los valores en forma binaria se utiliza la función "pasbin", la cual es igual a la que se utilizó en el primer ejemplo.

Las funciones "conv", "conv1" y "conv2" son funciones que realizan la impresión formateada de los datos en binario con la expresión de la misma.

ENUMERACIONES

Una enumeración es una estructura que permite asignar constantes literales a una variable entera. Sirve fundamentalmente para clarificar el código del programa fuente. No es posible imprimir o ingresar estas constantes a la variable desde un periférico de la computadora.

Debemos tener en cuenta que una variable declarada como enumeración es en sí misma de tipo entero, esto significa que esta se comporta como variable entera y por lo tanto puede recibir cualquier valor posible dentro del rango de los enteros, pero dentro del rango de la enumeración puede recibir ambos tanto la constante enumerada como el valor numérico correspondiente.

A la primera constante se le asigna el valor numérico cero por omisión y el resto continúa con el valor entero siguiente de acuerdo a su ubicación en la declaración. Se le puede asignar a la primera un valor distinto, utilizando el operador de asignación, pero las siguientes tomarán el valor desde el que tomó la primera incrementándose en una unidad de acuerdo a su ubicación.

Si se quiere utilizar un valor distinto al que posee una de las constantes intermedias se le puede asignar valores a éstas pero siempre deben ser superiores al que le correspondería.

Se la puede declarar de las siguientes formas:

```

enum etiqueta {constante_1, constante_2, constante_3, . . ., constante_n;}  
lista_variables;

```

```
enum { constante_1, constante_2, constante_3, . . ., constante_n;}  
lista_variables;
```

```
enum etiqueta { constante_1, constante_2, constante_3, . . ., constante_n};
```

De la misma forma que en una estructura permite tres diferentes formas de declararlas teniendo en cuenta que si no tiene la etiqueta de formato solo puede ser declarada una única vez.

Si se omite la declaración de la lista de las variables en la declaración se la debe realizar utilizando la palabra reservada "enum", a continuación la etiqueta del tipo y por último la lista de variables finalizada con ";".

```
enum etiqueta lista_variables;
```

Para asignarle un valor a la variable del tipo enumeración solo hace falta poner el nombre de la variable seguido del operador "=" y a continuación la constante válida terminada por ";".

EJEMPLO

```
#include<stdio.h>  
#include<conio.h>  
/* Definición de dos enumeraciones */  
enum dias {lunes,martes,miercoles,jueves,viernes,sabado,domingo};  
enum mes   {ene=20, feb, mar, abr=30, may, jun, jul=40,  
            \ago, sep, oct=50, nov, dic};  
void main(void)  
{  
    /* Declaración de las variables del tipo enum */  
    enum dias semana;  
    enum mes anio;  
    clrscr();  
    /* imprime datos dentro de los valores de la enumeración */  
    for(semana=lunes;semana<sabado;semana++)  
        printf("Días laborables %d\n",semana);  
    printf("\n");  
    for(anio=ene;anio<=dic;anio++)  
        printf("mes %d ",anio);  
    printf("\n\nValores posibles de semana\n");  
    /* Se asignan valores a la enumeración dentro del rango *  
    * declarado y fuera del mismo */  
    for(semana=-5;semana<=5;semana++)  
        printf("%d\t",semana);  
}
```

En este ejemplo se definen dos enumeraciones, la primera se define sin asignarle ningún valor previo a cada constante, y por lo tanto comenzará desde el valor cero. En la segunda se le asignan valores a ellas cambiando el valor de comienzo y algunos valores intermedios.

El programa imprime las variables dentro del rango definido mostrando su valor numérico, que es el que se puede utilizar.

En la ultima parte se ingresan valores a la variable enumerada dentro del rango de los enteros y también se puede observar que el funcionamiento es correcto.